

PA1 实验报告

231180166 杜明阳

实验进度

PA1实验任务全部完成，理解了图灵机TRM以及我们NEMU的基本架构，完成了简易调试器的所有要求指令，最后OJ所有测试用例都accepted.

运行部分sdb操作的结果打印如下

```
Welcome to riscv32-NEMU!
For help, type "help"
(nemu) si
0x80000000: 00 00 02 97 auipc    t0, 0

(nemu) p 10-5-2
[src/monitor/sdb/expr.c:110 make_token] match rules[7] = "[0-9]+" at position 0 with l
[src/monitor/sdb/expr.c:110 make_token] match rules[11] = "\"-\" at position 2 with l
[src/monitor/sdb/expr.c:110 make_token] match rules[7] = "[0-9]+" at position 3 with l
[src/monitor/sdb/expr.c:110 make_token] match rules[11] = "\"-\" at position 4 with l
[src/monitor/sdb/expr.c:110 make_token] match rules[7] = "[0-9]+" at position 5 with l
[src/monitor/sdb/expr.c:350 expr] Parsed 5 tokens:
[src/monitor/sdb/expr.c:352 expr] Token 0: type=260, str='10'
[src/monitor/sdb/expr.c:352 expr] Token 1: type=45, str='-'
[src/monitor/sdb/expr.c:352 expr] Token 2: type=260, str='5'
[src/monitor/sdb/expr.c:352 expr] Token 3: type=45, str='-'
[src/monitor/sdb/expr.c:352 expr] Token 4: type=260, str='2'
3

(nemu) p $pc
[src/monitor/sdb/expr.c:110 make_token] match rules[8] = "\\$[a-zA-Z][a-zA-Z0-9]*" a
[src/monitor/sdb/expr.c:350 expr] Parsed 1 tokens:
[src/monitor/sdb/expr.c:352 expr] Token 0: type=262, str='$pc'
2147483652
```

必答题

1. 程序是个状态机：画出计算 $1+2+\dots+100$ 的程序的状态机

```
#初始状态：
(0, x, x)
→ (1, 0, x)
→ (2, 0, 1)
→ (3, 1, 1)          # 第一次加法完成
#第一次循环：
→ (4, 1, 2)
→ (5, 1, 2)
→ (2, 1, 2)
→ (3, 3, 2)
#第二次循环：
→ (4, 3, 3)
→ (5, 3, 3)
→ (2, 3, 3)
→ (3, 6, 3)
...

#倒数第二次循环 (i=99)：
→ (4, 4950, 100)
→ (5, 4950, 100)
→ (2, 4950, 100)
→ (3, 5050, 100)
#最后一次循环：
→ (4, 5050, 101)
→ (5, 5050, 101)
→ (6, 5050, 101)    # 程序结束
```

2. 理解基础设施：简易调试器的时间节省计算

已知条件

- 总编译次数：500 次
- 调试占比：90% → 调试次数 = $500 \times 90\% = 450$ 次
- 每次调试中需获取信息的次数：假设为 1 次（题中未指定，按 1 次计算）
- GDB 方式：每次获取信息耗时 30 秒
- 简易调试器方式：每次耗时 10 秒

计算

1. 使用 GDB 的总调试时间：

2. $450 \times 30 \text{ 秒} = 13500 \text{ 秒} = 225 \text{ 分钟} = 3.75 \text{ 小时}$

3. 使用简易调试器的总调试时间：

4. $450 \times 10 \text{ 秒} = 4500 \text{ 秒} = 75 \text{ 分钟} = 1.25 \text{ 小时}$

5. 节省时间：

6. $13500 - 4500 = 9000$ 秒 = 150 分钟 = 2.5 小时

回答

如果没有实现简易调试器，仅通过 GDB 调试客户程序，本学期将在调试上花费 **3.75 小时**。
实现简易调试器后，调试时间降至 **1.25 小时**，共节省 **2.5 小时**。
实际中，每个 bug 可能需要获取多个信息，若每次调试需分析 10 个信息，则节省时间可达 **25 小时以上**，简易调试器的价值巨大。

3. RTFM：查阅 ISA 手册

(1) riscv32 有哪几种指令格式？

- 查阅手册：[The RISC-V Instruction Set Manual, Volume I: User-Level ISA](#)
- 位置：chapter2 2.2 Base Instruction Formats(R/I/S/U)2.3 Immediate Encoding Variants(B/J)
- 格式种类：
 - **R-type** : funct7[6:0] | rs2[4:0] | rs1[4:0] | funct3[2:0] | rd[4:0] | opcode[6:0]
 - **I-type** : imm[11:0] | rs1[4:0] | funct3[2:0] | rd[4:0] | opcode[6:0]
 - **S-type** : imm[11:5] | rs2[4:0] | rs1[4:0] | funct3[2:0] | imm[4:0] | opcode[6:0]
 - **B-type** : imm[12] | imm[10:5] | rs2[4:0] | rs1[4:0] | funct3[2:0] | imm[4:1] | imm[11] | opcode[6:0]
 - **U-type** : imm[31:12] | rd[4:0] | opcode[6:0]
 - **J-type** : imm[20] | imm[10:1] | imm[11] | imm[19:12] | rd[4:0] | opcode[6:0]

需阅读范围：**Chapter 2 2.2、2.3节**

(2) LUI 指令的行为是什么？

- 手册位置：Chapter 2 2.4节 Integer Computational Instructions
- 行为：LUI (load upper immediate) is used to build 32-bit constants and uses the U-type format. LUI places the U-immediate value in the top 20 bits of the destination register rd, filling in the lowest 12 bits with zeros.

```
rd = sext(immediate[31:12] << 12);
```

即：将 20 位立即数左移 12 位，符号扩展后写入目标寄存器。

需阅读范围：**volume1 Chapter 2 2.4jie**

(3) mstatus 寄存器的结构是怎么样的？

- 手册位置：Volume II, Chapter 3.1.6 Machine Status Registers (mstatus and mstatush)"

- **结构** (RISC-V Privileged Architecture) :
 - MIE (bit 3): Machine Interrupt Enable
 - MPIE (bit 7): Previous MIE
 - MPP (bits 12-11): Previous Privilege Mode
 - SIE, SPIE, SPP, UBE 等 (取决于扩展)
- **作用**：控制中断使能、特权模式切换。

需阅读范围：**Volume II, Chapter 3.1.6**

4. shell 命令：统计代码行数

(1) nemu/ 目录下所有 .c 和 .h 文件总行数

```
find nemu/ \( -name "*.c" -o -name "*.h" \) -exec cat {} + | wc -l
```

输出：267493

(2) PA1 新增代码行数

回到 pa0 分支 (PA1 之前)

```
git checkout pa0
```

统计当时代码行数

```
find nemu/ \( -name "*.c" -o -name "*.h" \) -exec cat {} + | wc -l # 记为 A
```

切回当前分支 (PA1 完成后)

```
git checkout your-pa1-branch
```

```
find nemu/ \( -name "*.c" -o -name "*.h" \) -exec cat {} + | wc -l # 记为 B
```

PA1 新增行数 = B - A

$A=266811$ newline $= B-A = 267493-266811=682$ 实际发现做了一个月了也没有写好多 (

(3) 除去空行的总行数

```
find nemu/ \( -name "*.c" -o -name "*.h" \) -exec cat {} + | grep -v "^$" | wc -l
```

输出231135

(5) 写入 Makefile

```
count:
@echo "Total lines (with blanks):"
@find nemu/ \( -name "*.c" -o -name "*.h" \) -exec cat {} + | wc -l
@echo "Total lines (no blanks):"
@find nemu/ \( -name "*.c" -o -name "*.h" \) -exec cat {} + | grep -v "^$$" | wc -l

clean-count:
@echo "PA1新增代码行数:"
@git diff pa0 HEAD nemu/ --name-only -z | xargs -0 cat | wc -l
```

使用：make count或者是make clean-count

5. RTFM：-Wall 和 -Werror 的作用

查阅手册：man gcc 或 [GCC Online Documentation](#)

(1) -Wall 的作用

- 启用“所有常见警告”（Warning All）
- 包括：
 - 未使用的变量
 - 未初始化变量
 - 函数未声明
 - 格式化字符串不匹配
 - 隐式类型转换等
- 目的：提前发现潜在 bug

(2) -Werror 的作用

- 将所有警告视为错误
- 编译时一旦有 warning，编译失败
- 强制开发者修复所有警告

(3) 为什么使用它们？

- 保证代码质量：避免“警告太多就无视”的恶性循环
- 团队协作：统一编码规范
- 防止隐患：如未初始化变量在模拟器中可能导致随机错误
- CI/CD 友好：自动化构建中不允许 warning

debug心得

在做表达式求职的时候，OJ hard test的一个始终通不过，尝试了括号匹配，空格过滤等等问题，最后发现是主运算符的判定有问题。

原来代码里面主运算符更新是这样写的 $\text{prec} < \text{min_prec}$ （从左到右找级别最高数值越低的op，如果有就更新 $\text{prec} \ \& \ \text{op}$ ），连大模型都是这样说的，说是这样可以实现左到右的匹配，即 $10-5-2=(10-5)-2$ ，主运算符就是第一个-。但是如果是递归求值BNF的话，主运算符应该是第二个-。即 $\text{prec} \leq \text{min_prec}$ 。

发现问题了就去反驳大模型为什么这样笨，在那里一直胡诌，他又开始道歉了..事实证明有时不能全靠大模型，还是自己要看的懂